

HLDD AI Agent

Agentic AI Implementation (LangGraph + Groq)

Generated: May 2026

Repository: github.com/jaiminbprajapatireminder-eng/SDLC-HLDD-AI

1. High-Level Architecture Overview

The HLDD AI Agent is a full-stack web application that ingests High-Level Design Documents, extracts structured project data, generates a delivery plan, and provides an interactive dashboard. It has two LLM-powered interfaces: a Legacy Chatbot (single-shot QA with 3-tier cascade) and a LangGraph Agentic AI (multi-turn ReAct loop with 5 tools).

This document covers ONLY the Agentic AI implementation.

Stack: React + Vite (frontend) | FastAPI (backend) | Groq (primary LLM) / Gemini (fallback LLM) | LangGraph (agent orchestration) | JIRA Cloud (project mgmt)

Application Component Diagram



2. What is the LangGraph Agent?

The LangGraph Agent replaces the previous single-shot chatbot with an autonomous AI agent that can reason, plan, and execute multi-step tasks using tools. Instead of following a fixed pipeline (parse -> plan -> respond), the agent uses a ReAct loop (Reasoning + Acting): it thinks about what to do next, calls a tool, gets the result, thinks again, and repeats until the task is complete.

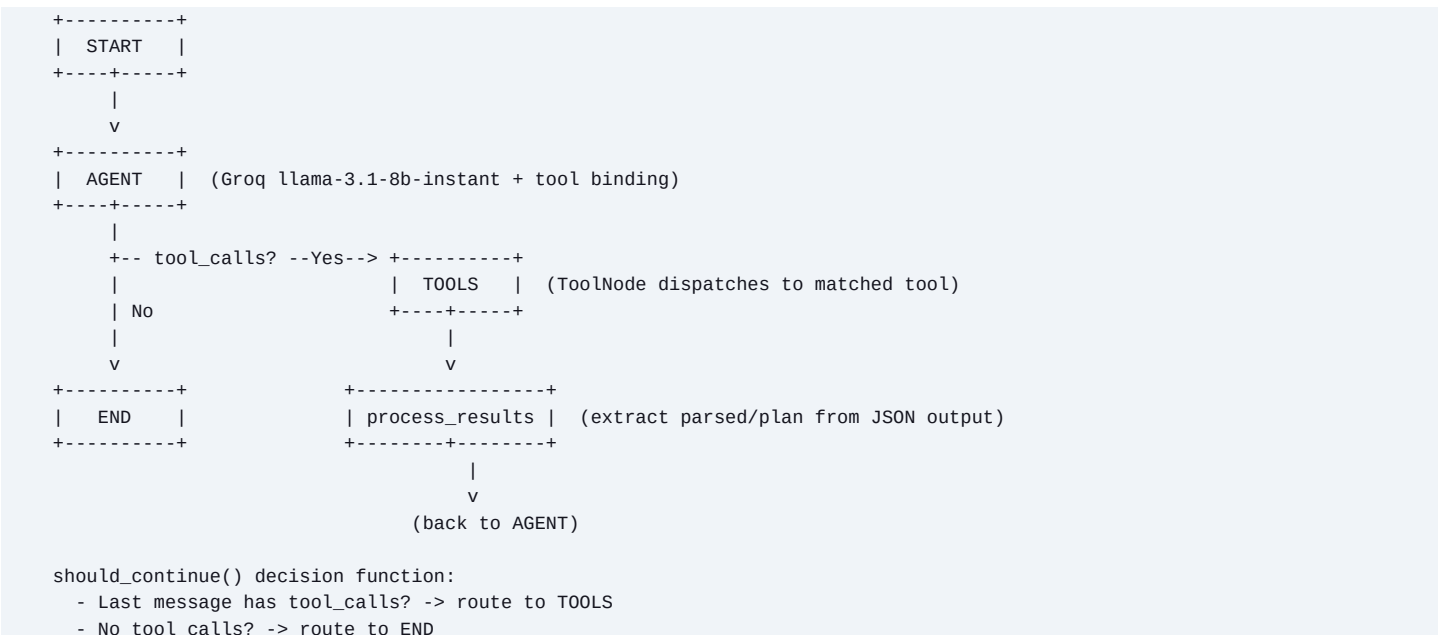
The agent uses Groq (llama-3.1-8b-instant) as its default LLM (with Gemini as fallback), bound to 5 tools that wrap the existing HLDD application logic. The frontend provides a dedicated Agent tab (separate from the Chatbot tab) with file upload, chat messages, and JIRA controls. Human-in-the-loop is enforced via the system prompt: the agent always asks for confirmation before creating or modifying JIRA items.

Comparison: Legacy Chatbot vs Agentic AI

LEGACY CHATBOT (Single-Shot): User question -> fixed pipeline -> resp POST /api/chatbot { prompt, priority } 1. is_out_of_scope() 2. Gemini dataflow (3 stages) 3. OpenAI fallback 4. Local rule templates Returns: { response, source }	AGENTIC AI (LangGraph): User goal -> agent decides -> tools -> loops -> done POST /api/agent { prompt, hldd_text, history, confirm } 1. Agent LLM (Groq llama-3.1-8b-instant + tools) 2. ToolNode (execute 1 of 5 tools) 3. Loop back to Agent LLM 4. END when task is done Returns: { response, state }
---	---

3. Agent Architecture

LangGraph StateGraph Structure



Agent State Schema (state.py)

AgentState = {	
raw_hldd_text: str	# Original HLDD document text
parsed_hldd: dict None	# Output of parse_hldd tool
project_plan: dict None	# Output of generate_plan tool
epic_mapping: dict	# feature_id -> JIRA epic key
story_mapping: dict	# story_id -> JIRA story key
messages: list	# Chat history (Human/AI/Tool msgs)
user_input: str	# Latest prompt from user
pending_confirmation: str	# Pending action awaiting user OK
}	

File Structure (backend/app/agent/)

```
backend/app/agent/  
+-- __init__.py      -- Package marker  
+-- state.py         -- AgentState TypedDict  
+-- prompts.py       -- AGENT_SYSTEM_PROMPT (~800 chars)  
+-- tools.py         -- 5 LangChain @tool functions  
+-- llm.py           -- LLM builder (Groq primary, Gemini fallback)  
+-- graph.py         -- StateGraph build + conditional edges
```

4. Technology Stack

Backend

- Python 3.9+ — FastAPI framework for REST API endpoints
- LangGraph — StateGraph agent orchestration with ReAct loop (Agent -> Tool -> Agent)
- LangChain Groq — ChatGroq integration for llama-3.1-8b-instant as the default agent LLM
- LangChain Google GenAI — ChatGoogleGenerativeAI fallback for Gemini 2.5-flash-lite
- LangChain Core — AIMessage, HumanMessage, ToolMessage types; ToolNode for tool dispatch
- Groq API — Primary LLM provider (llama-3.1-8b-instant, 30K TPD free tier)
- Google Gemini API — Fallback LLM provider (gemini-2.5-flash-lite)
- JIRA Cloud REST API v3 — Epics (10001), Stories (10004), Subtasks (10002)
- PyPDF2 / python-docx — Multi-format file parsing (.pdf, .docx)
- python-dotenv — Environment variable loading from .env file
- fpdf2 — PDF generation

Frontend

- React 18 — Component-based UI with hooks
- Vite — Build tool and dev server
- Mermaid.js — Architecture diagram rendering in the browser
- jsPDF + html2canvas — PDF export from DOM
- xlsx — Excel (.xlsx) export for Features, Stories, Testing tabs

5. LLM Configuration

Agent LLM Priority

```
Agent LLM (LangGraph) priority:
1. GROQ_API_KEY set      -> Groq llama-3.1-8b-instant (default)
2. GEMINI_API_KEY set    -> Gemini gemini-2.5-flash-lite (fallback)
3. Neither set           -> ValueError

Override: GROQ_MODEL env var (e.g. llama-3.3-70b-versatile for paid tiers)
```

Token Optimization for Groq Free Tier (6000 TPM limit)

- HLDD text truncated to 1500 characters before sending to the LLM
- Conversation history limited to last 6 messages to control cumulative token growth
- AI response content truncated to 500 characters per message
- Tool result content truncated to 600 characters per message
- Combined parse_and_generate_plan tool replaces two separate tools (parse_hldd + generate_plan)
- System prompt condensed from ~1900 to ~800 characters
- Reduced from 7 to 5 tools, cutting tool schema overhead by ~30%
- Graceful 429 HTTP response when rate limit is hit, with upgrade suggestion

GROQ_API_KEY Configuration

- Default model: llama-3.1-8b-instant (30,000 TPD free tier limit)
- Paid upgrade: llama-3.3-70b-versatile (set GROQ_MODEL env var)
- Fallback: GEMINI_API_KEY is used when GROQ_API_KEY is not set
- Rate limits: ~6000 TPM on free tier -- optimized via prompt truncation

6. Agent Prompts

6.1 AGENT_SYSTEM_PROMPT (master prompt)

This is the most important prompt -- it defines the agent's personality, capabilities, workflow rules, and human-in-the-loop requirements. It lives in `prompts.py`.

```
"""You are an HLDD Delivery Agent that analyzes High-Level Design
Documents and manages JIRA delivery.

## Tools
- parse_and_generate_plan: Parse HLDD and generate features/stories/tests.
- ask_gemini: Ask Gemini about the project, tech comparisons, or recommendations.
- create_jira_epic: Create a JIRA Epic for a feature (ask user first).
- create_jira_story: Create a JIRA Story linked to an Epic (ask user first).
- create_jira_subtask: Create a JIRA Subtask linked to a Story (ask user first).

## Rules
1. When user provides HLDD text, call parse_and_generate_plan. Report results.
2. For JIRA: only create when user explicitly asks. Always ask confirmation first.
3. For questions: use ask_gemini.
4. Be concise. If no HLDD provided, ask the user to provide one.
"""
```

6.2 Tool Docstrings (per-tool prompts)

Each `@tool` function's docstring IS a prompt -- it tells the LLM when to call the tool, what inputs it needs, and what it returns. The LLM reads these docstrings at inference time via Gemini's function calling / tool binding.

6.3 Context Injection (dynamic prompt)

Before each agent turn, the `agent_node` function injects the current HLDD context into the system message. This is built dynamically from the state:

```
## Current HLDD Context
Title: {title}
Tech Stack: React, FastAPI, PostgreSQL
Functional Requirements: 4

## Current Project Plan
Features: 4
Development Stories: 12
Testing Stories: 12
JIRA Epics Created: 2 (SCRUM-50, SCRUM-51)
JIRA Stories Created: 0
```

7. Tool Definitions

`parse_and_generate_plan(raw_hldd_text: str) -> dict`

- Wraps `parser.parse_hldd_document()` then `generator.generate_project_plan()` in one step. Extracts title, functional_requirements, technology_stack, and immediately generates features, stories, testing stories, architecture diagram, and project structure. Preferred over calling `parse_hldd` + `generate_plan` separately.
- Wraps `gemini_dataflow.call_gemini_dataflow()`. For general Q&A, technology comparisons, cost analysis, architecture advice, risk assessment. Always passes current HLDD context.
- Creates an Epic in JIRA SCRUM project. Returns JIRA key (e.g., SCRUM-42). Must be called BEFORE `create_jira_story`. Human confirmation required.
- Creates a Story in JIRA linked to a parent Epic via the parent field. Requires `epic_key` from `create_jira_epic`. Human confirmation required.
- Creates a Subtask in JIRA linked to a parent Story. Requires `parent_key` from `create_jira_story`. Human confirmation required.

8. Agent Execution Flow

Example: Parse HLDD and Create JIRA Epics

```

Step 1: Agent LLM receives system prompt + user message
  Thinks: 'I need to parse the HLDD and generate a plan'
  Calls: parse_and_generate_plan(raw_text)
  State: parsed_hldd = { title, tech_stack, ... }
         project_plan = { features: [...], stories: [...] }

Step 2: process_tool_results extracts parsed_hldd and project_plan
  from the ToolMessage JSON output, then loops back to Agent

Step 3: Agent LLM sees plan. User asked for JIRA.
  Thinks: '4 features found. Must ask user before creating.'
  Responds: 'I found 4 features. Shall I create 4 Epics in JIRA?'
  Pauses (waits for user input)

Step 4: User says 'Yes, proceed'
  Agent: Calls create_jira_epic() x 4
  Results: SCRUM-50, SCRUM-51, SCRUM-52, SCRUM-53
  State: epic_mapping = { IPMA#001: SCRUM-50, ... }

Step 5: Agent reports results
  Responds: 'Created 4 Epics: SCRUM-50 through SCRUM-53'
  END

```

Graph Transitions

```

Agent Node -> should_continue()
|
+-- Last msg has tool_calls? --> Tools Node
|
|                               |
|                               +-- Execute tool
|                               +-- Return result as ToolMessage
|                               +-- Go back to Agent Node
|
+-- Last msg has NO tool_calls? --> END
|
+-- Return final response

```

9. Human-in-the-Loop Design

- The system prompt instructs the agent to ALWAYS ask before JIRA actions
- The LLM generates the confirmation question naturally (no code-level interrupt)
- User responds with 'yes'/'proceed'/'confirm' -- agent then calls JIRA tools
- If user says 'no'/'cancel'/'stop' -- agent does NOT call JIRA tools and asks what next
- The confirm field in the AgentRequest allows frontend to pass explicit approval
- All 5 tools are pure functions -- they cannot modify state without being called

10. FastAPI Integration

Endpoint: POST /api/agent

```
Request:
{
  'prompt': 'Parse this HLDD and create Epics for all features',
  'hldd_text': 'HLDD document content...',
  'history': [
    # conversation so far
    { 'role': 'user', 'content': '...' },
    { 'role': 'assistant', 'content': '...' }
  ],
  'confirm': false # set true to confirm pending action
}

Response:
{
  'response': 'Created 4 Epics: SCRUM-50, SCRUM-51...',
  'state': {
    'parsed': { ... }, # parsed HLDD data
    'plan': { ... }, # project plan
    'epics': { IPMA#001: SCRUM-50, ... },
    'stories': {}
  }
}
```

Error Handling

- If GROQ_API_KEY and GEMINI_API_KEY are both unset: HTTP 400 with clear error
- If JIRA_EMAIL/JIRA_API_TOKEN not set: JIRA tools return error string (not crash)
- Tool failures are returned as ToolMessage content, fed back to agent LLM
- Agent LLM reads the error and asks the user how to proceed
- Groq rate limits: HTTP 429 with retry suggestion and upgrade link

State Flow per Request

1. Frontend sends { prompt, hldd_text, history, confirm }
2. Backend converts history to LangChain message objects
3. Creates AgentState: raw_hldd_text, messages + user_input
4. Invokes agent (StateGraph) with recursion_limit=50
5. Graph loops: Agent Node -> Tools -> Agent -> ... -> END
6. Last message content returned as 'response'
7. parsed_hldd, project_plan, epic_mapping, story_mapping extracted from state and returned

11. File Upload via Agent Chat

The frontend Agent tab provides a file upload input and a chat interface. Users can upload .txt, .md, .docx, or .pdf HLDD documents directly to the agent without using the main /api/upload endpoint.

Upload Flow

1. User clicks file input in agent chat
2. Frontend reads file as text (FileReader / .text())
3. Frontend calls POST /api/agent with:

```
{ prompt: 'Parse this HLDD...', hldd_text: '<text>', history: [] }
```
4. Agent LLM receives file content as hldd_text
5. Agent decides to call parse_and_generate_plan(raw_text) tool
6. Returns structured sections + project plan in one call
7. Agent reports results to user in chat
8. User can continue: 'Create Epics in JIRA', 'Summarize the plan', etc.

Supported Formats

Format	Method

.txt	file.text() in browser (client-side)
.md	file.text() in browser (client-side)
.docx	POST /api/extract-text (python-docx)
.pdf	POST /api/extract-text (PyPDF2)

Example Prompts for the Agent

Prompt	Agent Action

'Upload an HLDD file and parse it'	Prompts user to upload
'What can you do?'	Lists capabilities
'Summarize the delivery plan'	Summarizes parsed plan
'Create Epics in JIRA for all features'	Creates Epics (asks first)
'Create Stories linked to the Epics'	Creates Stories
'Create Subtasks for all Stories'	Creates Subtasks
'Compare AWS vs GCP for this project'	Calls ask_gemini tool
'What are the risks?'	Risk analysis via Gemini
'Yes' / 'Proceed'	Confirms pending JIRA action
'No' / 'Cancel'	Cancels pending action

12. RAG (Retrieval-Augmented Generation)

RAG is IMPLEMENTED as of May 2026. The application uses ChromaDB (ephemeral by default) with a DefaultEmbeddingFunction to index HLDD document chunks for vector search.

Architecture

Upload Flow:

1. User uploads HLDD (.txt/.md/.docx/.pdf)
2. Text is extracted via `_read_upload()`
3. Text is chunked (500 chars, 50 overlap) and indexed in ChromaDB
4. `retriever.ingest(raw_text)` deletes old chunks, inserts new ones

Retrieval Flow (Agent tool):

1. Agent calls `retrieve_hldd_context(question)`
2. ChromaDB performs vector similarity search (k=3)
3. Top-k chunks returned as plain text
4. Agent uses retrieved context instead of truncated text

Components

- `backend/app/rag.py` — HLDDRetriever class (ingest, retrieve, clear)
- ChromaDB — vector store (ephemeral in-memory, configurable to persistent)
- DefaultEmbeddingFunction — built-in ChromaDB embedding (no extra model needed)
- Chunk size: 500 chars with 50 char overlap
- Agent tool: `retrieve_hldd_context(question)` -> relevant chunks
- Wired into upload flow: `retriever.ingest()` called after every upload

13. MCP (Model Context Protocol)

MCP is IMPLEMENTED as of May 2026. The application exposes an HTTP-based MCP endpoint and an agent tool to call external MCP tools. The MCP server can also run as a standalone stdio process via `python -m app.mcp_server`.

Architecture

```
HTTP Endpoint (POST /mcp/call):
Request: { tool, arguments }

Available MCP tools:
- parse_hldd(text)      -> parse HLDD into structured data
- generate_plan(data)   -> generate project plan from parsed data
- ask_chatbot(question, context, priority) -> ask the legacy chatbot

Agent Integration:
Agent tool: mcp_call(tool_name, arguments) -> calls POST /mcp/call
The LangGraph agent can invoke MCP tools as part of its ReAct loop
```

Components

- backend/app/mcp_server.py — Standalone MCP Server (stdio, no external deps)
 - POST /mcp/call — FastAPI endpoint for HTTP MCP calls
 - Agent tool: mcp_call(tool_name, arguments) -> str
 - 3 tools: parse_hldd, generate_plan, ask_chatbot
 - Start standalone: `python -m app.mcp_server`
-
- No true RAG -- context stuffing limits HLDD document size to ~1500 chars
 - Groq free tier limited to 6000 TPM / 30,000 TPD -- may hit rate limits on heavy usage
 - No streaming -- agent response returns as a single block (no SSE)
 - No persistent state -- AgentState is created per-request, conversation lives in frontend history
 - No database -- LATEST_PAYLOAD is in-memory, lost on backend restart
 - No auth/authentication -- open endpoints, CORS allows all origins
 - JIRA project hardcoded to 'SCRUM' at mylearningdata.atlassian.net
 - Recursion limit set to 50 -- infinite loops are bounded but long chains fail silently

Variable	Required	Description

GROQ_API_KEY	Preferred	Groq key for agent LLM (llama-3.1-8b-instant)
GEMINI_API_KEY	Fallback	Gemini key (fallback when GROQ_API_KEY not set)
JIRA_EMAIL	For JIRA	JIRA Cloud email
JIRA_API_TOKEN	For JIRA	JIRA Cloud API token
GROQ_MODEL	No	Override default Groq model

15. Future Enhancements

- Add `interrupt_before=['tools']` in `graph.compile()` for code-level human-in-the-loop (pauses before every tool)
- Switch ChromaDB to persistent mode and add sentence-transformers for better embeddings
- Add `web_search` MCP tool for real-time technology/cost research
- Add MCP stdio client for IDE tool access (file editing, git operations)
- Add streaming response via Server-Sent Events so the agent's reasoning is visible in real-time
- Add persistent state via SQLite checkpointing (LangGraph's built-in MemorySaver)
- Replace Gemini with a fine-tuned model specialized in HLDD analysis